

AI Assisted Coding

Assignment 16.3

Name: P. Vineeth Kumar

Hall ticket no: 2303A51256

Batch no: 19

Task 1:

Prompt:

Design a Hospital Management database in SQLite by creating three related tables: Doctors, Patients, and Appointments. The tables should include primary keys, foreign keys, NOT NULL constraints, and unique constraints where necessary. Write SQL queries using joins to display all appointments for a specific doctor, retrieve the full history of a patient, and count the number of distinct patients treated by each doctor. Insert sample data and include SELECT queries to verify the database.

Code & Output:

```
1  # Task- 1 :
2  DROP TABLE IF EXISTS Appointments;
3  DROP TABLE IF EXISTS Doctors;
4  DROP TABLE IF EXISTS Patients;
5  CREATE TABLE Doctors (
6      doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
7      first_name TEXT NOT NULL,
8      last_name TEXT NOT NULL,
9      specialty TEXT NOT NULL
10 );
11 CREATE TABLE Patients (
12     patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
13     first_name TEXT NOT NULL,
14     last_name TEXT NOT NULL,
15     date_of_birth TEXT NOT NULL
16 );
17 CREATE TABLE Appointments (
18     appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
19     doctor_id INTEGER,
20     patient_id INTEGER,
21     appointment_date TEXT,
22     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
23     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
24 );
25 INSERT INTO Doctors (first_name, last_name, specialty)
26 VALUES ('Rahul', 'Sharma', 'Cardiology'),
27         ('Anita', 'Reddy', 'Neurology');
28
29 INSERT INTO Patients (first_name, last_name, date_of_birth)
30 VALUES ('Kiran', 'Kumar', '1995-06-10'),
31         ('Priya', 'Rao', '2000-03-21');
32 INSERT INTO Appointments (doctor_id, patient_id, appointment_date)
33 VALUES (1,1,'2024-07-01'),
34         (1,2,'2024-07-02'),
35         (2,1,'2024-07-03');
36 SELECT * FROM Doctors;
37 SELECT * FROM Patients;
38 SELECT * FROM Appointments;
39 SELECT a.appointment_id, d.first_name, p.first_name, a.appointment_date
40 FROM Appointments a
41 JOIN Doctors d ON a.doctor_id = d.doctor_id
42 JOIN Patients p ON a.patient_id = p.patient_id
43 WHERE a.doctor_id = 1;
44 SELECT p.first_name, p.last_name, a.appointment_date
45 FROM Appointments a
46 JOIN Patients p ON a.patient_id = p.patient_id
47 WHERE p.patient_id = 1;
48 SELECT d.first_name, COUNT(DISTINCT a.patient_id)
49 FROM Doctors d
50 LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
51 GROUP BY d.doctor_id;
```

Output 1 – Doctors Table			
doctor_id	first_name	last_name	specialty
1	Rahul	Sharma	Cardiology
2	Anita	Reddy	Neurology
Output 2 – Patients Table			
patient_id	first_name	last_name	date_of_birth
1	Kiran	Kumar	1995-06-10
2	Priya	Rao	2000-03-21
Output 3 – Appointments Table			
appointment_id	doctor_id	patient_id	appointment_date
1	1	1	2024-07-01
2	1	2	2024-07-02
3	2	1	2024-07-03
Output 4 – Appointments for Doctor			
appointment_id	doctor_name	patient_name	date
1	Rahul	Kiran	2024-07-01
2	Rahul	Priya	2024-07-02

Explanation:

This database is designed using three related tables connected through foreign keys to maintain relationships between doctors, patients, and appointments. SQL joins are used to retrieve appointment details and patient history efficiently. Aggregate functions are used to count the number of patients treated by each doctor. The database structure follows normalization to avoid redundancy and maintain data integrity.

Task 2:

Prompt:

Create an E-commerce database system using SQLite with tables Users, Products, Orders, and OrderDetails. Include primary keys, foreign keys, and necessary constraints. Write SQL queries to retrieve orders placed by a specific user, identify the most purchased product, and calculate total revenue for a particular month. Insert sample data and verify using SELECT queries.

Code & Output:

```
53 # Task- 2:
54 DROP TABLE IF EXISTS OrderDetails;
55 DROP TABLE IF EXISTS Orders;
56 DROP TABLE IF EXISTS Products;
57 DROP TABLE IF EXISTS Users;
58 CREATE TABLE Users (
59     user_id INTEGER PRIMARY KEY AUTOINCREMENT,
60     username TEXT NOT NULL,
61     email TEXT UNIQUE
62 );
63 CREATE TABLE Products (
64     product_id INTEGER PRIMARY KEY AUTOINCREMENT,
65     product_name TEXT NOT NULL,
66     price REAL NOT NULL,
67     stock INTEGER NOT NULL
68 );
69 CREATE TABLE Orders (
70     order_id INTEGER PRIMARY KEY AUTOINCREMENT,
71     user_id INTEGER,
72     order_date TEXT,
73     FOREIGN KEY (user_id) REFERENCES Users(user_id)
74 );
75 CREATE TABLE OrderDetails (
76     order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
77     order_id INTEGER,
78     product_id INTEGER,
79     quantity INTEGER,
80     price REAL,
81     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
82     FOREIGN KEY (product_id) REFERENCES Products(product_id)
83 );
84 INSERT INTO Users (username, email) VALUES ('charan', 'charan@gmail.com');
85 INSERT INTO Products (product_name, price, stock)
86 VALUES ('Laptop', 50000, 5),
87         ('Mouse', 500, 20);
88 INSERT INTO Orders (user_id, order_date) VALUES (1, '2024-01-10');
89 INSERT INTO OrderDetails (order_id, product_id, quantity, price)
90 VALUES (1, 1, 1, 50000),
91         (1, 2, 2, 500);
92 SELECT * FROM Users;
93 SELECT * FROM Products;
94 SELECT * FROM Orders;
95 SELECT * FROM OrderDetails;
96 SELECT o.order_id, od.product_id, od.quantity
97 FROM Orders o
98 JOIN OrderDetails od ON o.order_id = od.order_id
99 WHERE o.user_id = 1;
100 SELECT p.product_name, SUM(od.quantity)
101 FROM Products p
102 JOIN OrderDetails od ON p.product_id = od.product_id
103 GROUP BY p.product_id
104 ORDER BY SUM(od.quantity) DESC
105 LIMIT 1;
106 SELECT SUM(od.quantity * od.price)
107 FROM Orders o
108 JOIN OrderDetails od ON o.order_id = od.order_id
109 WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

Users Table		
user_id	username	email
1	charan	charan@gmail.com

Products Table			
product_id	product_name	price	stock
1	Laptop	50000	5
2	Mouse	500	20

Orders Table		
order_id	user_id	order_date
1	1	2024-01-10

OrderDetails Table				
order_detail_id	order_id	product_id	quantity	price
1	1	1	1	50000
2	1	2	2	500

Most Purchased Product Output	
product_name	total_quantity
Mouse	2

Explanation:

The E-commerce database consists of multiple related tables where OrderDetails connects Orders and Products, maintaining proper normalization. SQL joins are used to retrieve order information and analyze product sales. Aggregate functions help identify the most purchased product and calculate total revenue. This structure avoids data duplication and improves query efficiency.

Task 3:

Prompt:

Design a Library Management database using SQLite with three tables: Books, Members, and Loans. Include primary keys, foreign keys, and constraints such as NOT NULL and UNIQUE. Write SQL queries to retrieve books that are currently issued, find overdue books where the loan period is more than 30 days, and count the number of books borrowed by each member. Insert sample data and ensure the database is normalized.

Code & Output:

```
111 # Task- 3:
112 DROP TABLE IF EXISTS Loans;
113 DROP TABLE IF EXISTS Books;
114 DROP TABLE IF EXISTS Members;
115 CREATE TABLE Books (
116     BookID INTEGER PRIMARY KEY AUTOINCREMENT,
117     Title TEXT NOT NULL,
118     Author TEXT NOT NULL,
119     PublishedYear INTEGER
120 );
121 CREATE TABLE Members (
122     MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
123     FirstName TEXT NOT NULL,
124     LastName TEXT NOT NULL,
125     Email TEXT UNIQUE
126 );
127 CREATE TABLE Loans (
128     LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
129     BookID INTEGER,
130     MemberID INTEGER,
131     LoanDate DATE,
132     ReturnDate DATE,
133     FOREIGN KEY (BookID) REFERENCES Books(BookID),
134     FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
135 );
136 INSERT INTO Books (Title, Author, PublishedYear)
137 VALUES ('Database Systems', 'Navathe', 2018),
138         ('Operating Systems', 'Galvin', 2016);
139 INSERT INTO Members (FirstName, LastName, Email)
140 VALUES ('Ravi', 'Kumar', 'ravi@gmail.com'),
141         ('Sneha', 'Reddy', 'sneha@gmail.com');
142 INSERT INTO Loans (BookID, MemberID, LoanDate)
143 VALUES (1,1,'2024-05-01'),
144         (2,2,'2024-05-15');
145 SELECT b.Title, m.FirstName, l.LoanDate
146 FROM Loans l
147 JOIN Books b ON l.BookID = b.BookID
148 JOIN Members m ON l.MemberID = m.MemberID
149 WHERE l.ReturnDate IS NULL;
150 SELECT b.Title, l.LoanDate
151 FROM Loans l
152 JOIN Books b ON l.BookID = b.BookID
153 WHERE julianday('now') - julianday(l.LoanDate) > 30;
154 SELECT m.FirstName, COUNT(l.LoanID)
155 FROM Members m
156 LEFT JOIN Loans l ON m.MemberID = l.MemberID
157 GROUP BY m.MemberID;
```

Books Issued		
Title	Member	LoanDate
Database Systems	Ravi	2024-05-01
Operating Systems	Sneha	2024-05-15

Overdue Books	
Title	LoanDate
Database Systems	2024-05-01

Books Loaned Per Member	
Member	BooksLoaned
Ravi	1
Sneha	1

Explanation:

The library database is designed using three related tables connected through foreign keys. SQL joins are used to retrieve issued books, overdue books, and borrowing details. Aggregate functions help calculate the number of books borrowed by each member. The database structure follows normalization principles to reduce redundancy and improve data integrity.

Task 4:

Prompt:

Analyze and optimize SQL queries for better performance. First write a query using a subquery to retrieve books written by UK authors, then rewrite the same query using a JOIN for optimization. Ensure both queries return the same result and suggest indexing to improve performance.

Code & Output:

```

159 # Task- 4:
160 DROP TABLE IF EXISTS Books;
161 DROP TABLE IF EXISTS Authors;
162 CREATE TABLE Authors (
163     id INTEGER PRIMARY KEY AUTOINCREMENT,
164     name TEXT,
165     country TEXT
166 );
167 CREATE TABLE Books (
168     book_id INTEGER PRIMARY KEY AUTOINCREMENT,
169     title TEXT,
170     author_id INTEGER,
171     FOREIGN KEY (author_id) REFERENCES Authors(id)
172 );
173 INSERT INTO Authors (name, country)
174 VALUES ('Author A', 'UK'),
175         ('Author B', 'USA'),
176         ('Author C', 'UK');
177 INSERT INTO Books (title, author_id)
178 VALUES ('Book One', 1),
179         ('Book Two', 2),
180         ('Book Three', 3);
181 -- Original Query (Subquery)
182 SELECT title
183 FROM Books
184 WHERE author_id IN (
185     SELECT id FROM Authors WHERE country = 'UK'
186 );
187 -- Optimized Query (JOIN)
188 SELECT b.title
189 FROM Books b
190 JOIN Authors a ON b.author_id = a.id
191 WHERE a.country = 'UK';
192 -- Indexing
193 CREATE INDEX idx_books_author_id ON Books(author_id);
194 CREATE INDEX idx_authors_country ON Authors(country);
195

```

title
Book One
Book Three
Both original query and optimized query should return same result.

Explanation:

The original query uses a subquery which may scan the database multiple times and reduce performance. The optimized query uses a JOIN operation which improves execution speed and scalability. Indexing important columns such as author_id and country further improves query performance. Both queries return the same results but the optimized query executes faster.

Task 5:

Prompt:

Design a University Course Registration database with four tables: Students, Faculty, Courses, and Registrations. Include primary keys, foreign keys, and constraints. Write SQL queries to list students enrolled in a course, find faculty teaching more than two courses, and retrieve the course with the highest number of registrations.

Code & Output:

Explanation:


```

196 # Task -5 :
197 DROP TABLE IF EXISTS Registrations;
198 DROP TABLE IF EXISTS Courses;
199 DROP TABLE IF EXISTS Faculty;
200 DROP TABLE IF EXISTS Students;
201 CREATE TABLE Students (
202     student_id INTEGER PRIMARY KEY AUTOINCREMENT,
203     first_name TEXT,
204     last_name TEXT,
205     email TEXT UNIQUE
206 );
207 CREATE TABLE Faculty (
208     faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
209     first_name TEXT,
210     last_name TEXT,
211     email TEXT UNIQUE
212 );
213 CREATE TABLE Courses (
214     course_id INTEGER PRIMARY KEY AUTOINCREMENT,
215     course_name TEXT,
216     faculty_id INTEGER,
217     FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
218 );
219 CREATE TABLE Registrations (
220     registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
221     student_id INTEGER,
222     course_id INTEGER,
223     FOREIGN KEY (student_id) REFERENCES Students(student_id),
224     FOREIGN KEY (course_id) REFERENCES Courses(course_id)
225 );
226 INSERT INTO Faculty (first_name, last_name, email)
227 VALUES ('Dr', 'Rao', 'rao@university.com');
228 INSERT INTO Students (first_name, last_name, email)
229 VALUES ('Charan', 'Yadav', 'charan@gmail.com');
230 INSERT INTO Courses (course_name, faculty_id)
231 VALUES ('Database Systems', 1);
232 INSERT INTO Registrations (student_id, course_id)
233 VALUES (1,1);
234 SELECT s.first_name, s.last_name
235 FROM Students s
236 JOIN Registrations r ON s.student_id = r.student_id
237 WHERE r.course_id = 1;
238 SELECT f.first_name, COUNT(c.course_id)
239 FROM Faculty f
240 JOIN Courses c ON f.faculty_id = c.faculty_id
241 GROUP BY f.faculty_id
242 HAVING COUNT(c.course_id) > 1;
243 SELECT c.course_name, COUNT(r.registration_id)
244 FROM Courses c
245 JOIN Registrations r ON c.course_id = r.course_id
246 GROUP BY c.course_id
247 ORDER BY COUNT(r.registration_id) DESC
248 LIMIT 1;

```

Students in Course

first_name	last_name
Charan	Yadav

Course with Highest Registration

course_name	registration_count
Database Systems	1

Explanation:

The university course registration database uses multiple related tables where the Registrations table connects Students and Courses to manage many-to-many relationships. SQL joins are used to retrieve enrollment details and faculty course information. Aggregate functions help identify popular courses and faculty workload. The database structure is normalized to maintain consistency and reduce data redundancy.